

Sigma

SIEM Detection Format

# Detection Engineering with Sigma

Thomas Patzke

# Agenda



1. Introduction into Sigma
2. How rules and correlation rules look like?
3. Best practices in rule writing
4. Using LLMs to create rules
5. Introducing some tools
6. Hands-on until the time is over

Hands-on:

- A text editor with YAML support.
  - Recommendation: VSCode with Sigma extension.
- A conversion tool
  - Sigma CLI
  - Sigconverter.io (integrated within VSCode Sigma extension)
- A clone of the workshop repository.

https://github.com/thomaspatzke/detection-engineering-with-sigma



# Introduction

# Why Using Sigma Rules?



- Not reinventing things: repository of 3k+ existing open source rules, steadily growing.
- Write-once, use many times: many organizations don’t have a single target system but various (SIEM, EDR, data lake, migrations, M&A …)
- Reduce vendor lock-in: switch your SIEM just by changing the conversion configuration instead of migrating every single detection.
- Share generic detection logic instead of vendor-specific queries.

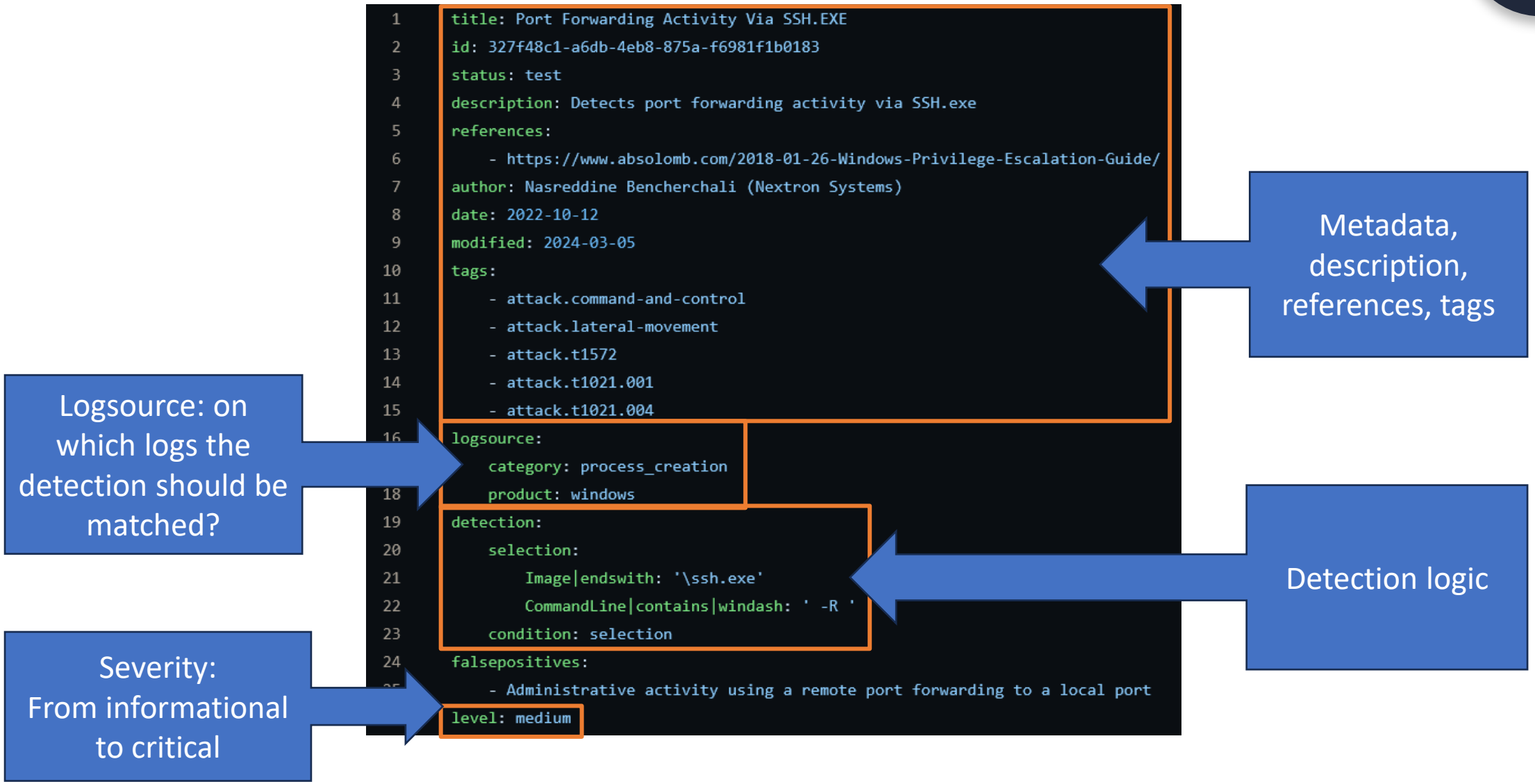
# Sigma Goals and Scope



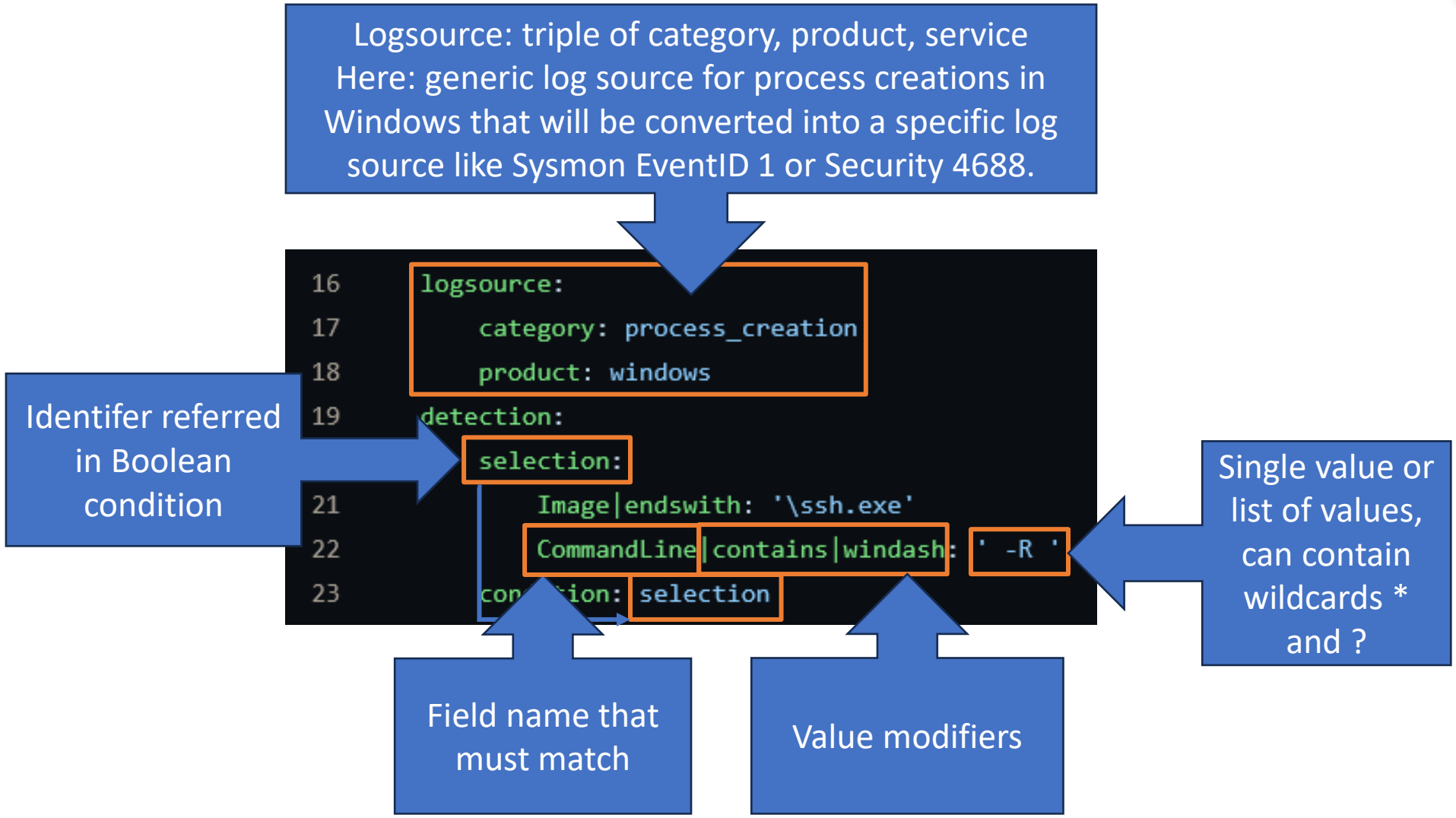
- Rules should be easily writable.
- Human-readable actionable rules.
- Machine-readable: toolchain for conversion into queries or SIEM/EDR rules.
- Simple: expressiveness for 95% of detections, not every imaginable use case.
- Generic: detections written in Sigma should be convertible into most target languages.
- Not just detection logic but also context and metadata.
- Shareable

# Sigma Rules

# Anatomy of a Sigma Rule



# Logsource & Detection

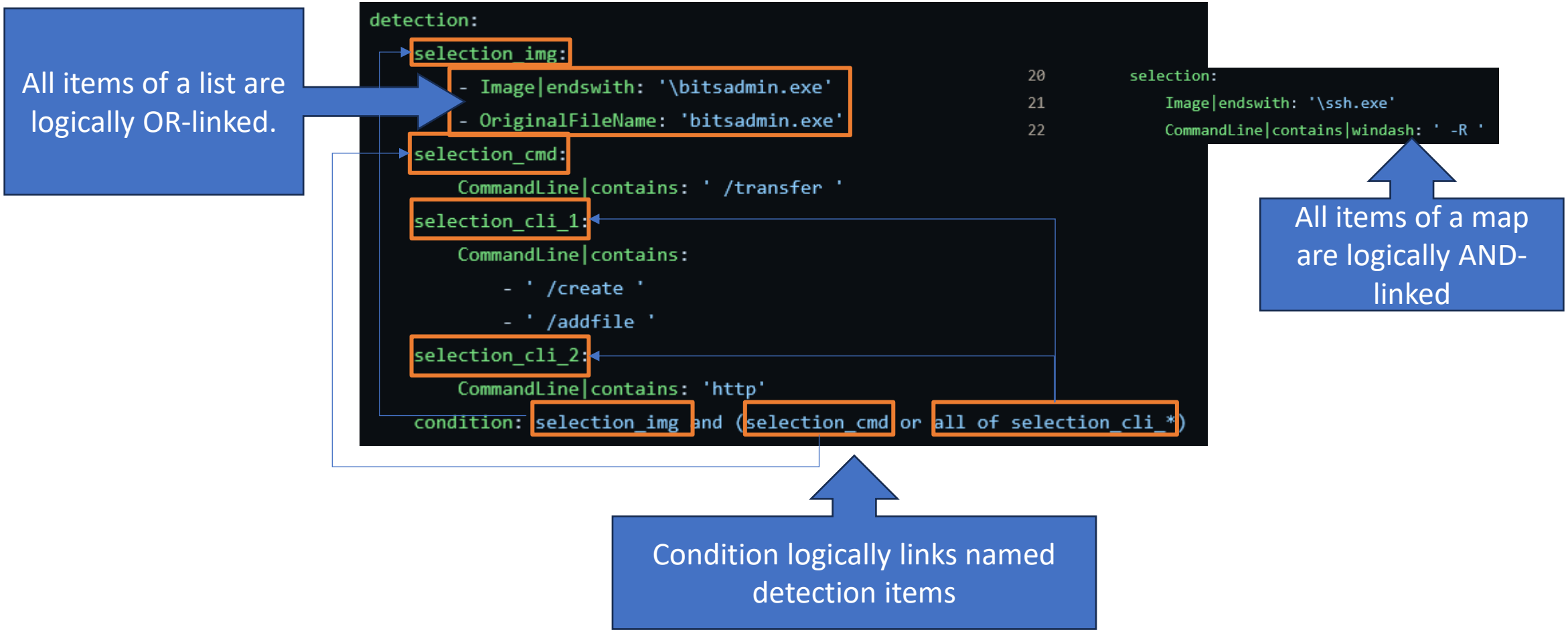


# Value Modifiers



| Modifier | Purpose |
|---|---|
| startswith/endswith/contains | Value starts, ends or contains the given value (instead of *value*) |
| re | Match value as regular expression. |
| cidr | Match value as CIDR network range (e.g. 10.0.0.0/8) |
| base64 | Match given value Base64-encoded as whole. |
| base64offset\|contains | Match given value as part of base64 encoded string. |
| wide/utf16/utf16le | Convert to given encoding. |
| windash | Expand parameter dashes “–” into all forms that are also valid like /. |
| gt/gte/lt/lte | >, >=, <, <= on numerical values (e.g. HTTP status codes) |
| exists | Check for field existence, value is yes or no. |
| expand | Expand %placeholders% as configured in processing pipelines. |
| fieldref | Reference another field for comparison |
| all | AND-link list elements instead of default OR-linking. |

# Logic & Condition



# Rule Conventions



- File name should be <log_source>_<short detection title>.yml, e.g.
  - win_security_susp_add_sid_history.yml
  - proc_creation_win_rundll32_uncommon_dll_extension.yml
  - proc_creation_win_susp_double_extension.yml
- Rule attributes should have an order, typically they start with the metadata followed by the detection logic and closing with false positives and level.

# Best Practices



- Avoid keyword rules, always try to match named fields.
- Avoid regular expressions, they are less performant in many SIEMs.
- Try to put wildcards at token-boundaries: `*\path\*` has usually a much better performance as `*path*`.
- Use modifiers instead of wildcards.
- Double escape backslashes in double quote YAML context or generally.
- Use Sigma validation with “sigma check”.
- Avoid complexity! Rule of thumb: additional complexity in detection rules should provide value (often less false positives). Examples:
  - Add only conditions if they reduce false positives.
  - Write only correlation rules if detecting the single event is not sufficient.

# Sigma Correlation Rules

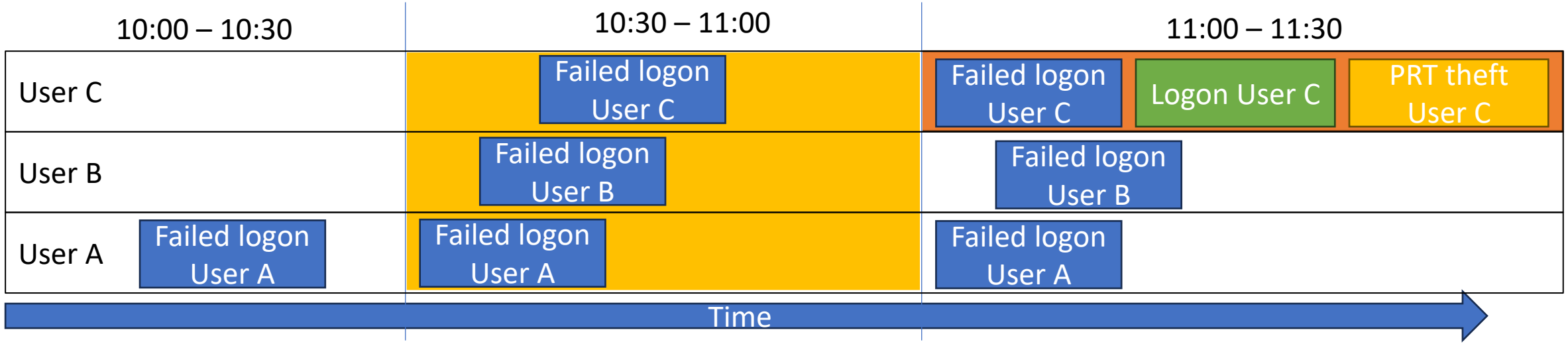
# Sigma Correlation Rules



- Sigma rules match on *single events* that match the defined conditions.
- Some detections need further logic that takes the context of such single events into account.
- Example: brute force or password spraying attacks are characterized by multiple attempts for guessing a password.
- Sigma correlation rules provide a framework to express such relationships between single events in a generic way.

# Correlation Concepts

- A correlation rule matches on *base events* of different *event types*.
- Correlation rules detect across a given *time frame*.
- Single events can be *grouped* by commonalities, e.g. they appear for the same user.
- Time frame and grouping define a *correlation scope*.
- The *correlation condition* defines how the events must appear within the correlation scope to trigger the detection.
- There are different *correlation types*.

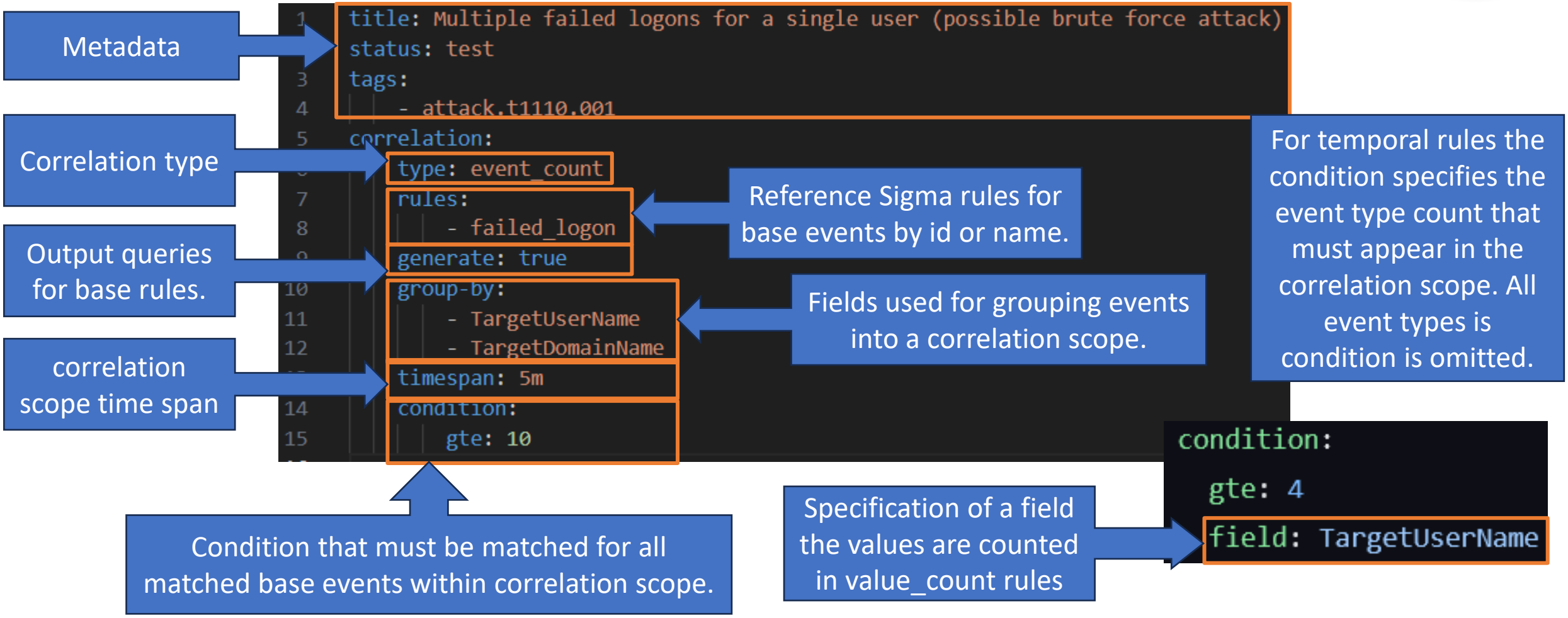


# Correlation Types



| Type | Description |
|---|---|
| event_count | Event count matches a condition in the correlation scope.<br>**Example:** 10 failed logons for a user in 10 minutes = brute force attempt |
| value_count | Within a given correlation scope the count of different values for a given field matches a condition.<br>**Example:** Failed logons for 10 different users by the same source IP = password spray |
| temporal | Within the correlation scope events of different type are observed with at least the given count of types.<br>**Example:** failed and successful logon and a PRT token theft by the same user |
| ordered_temporal | Like temporal, but the event types furthermore appear in the given order. |
| value_sum<br>value_avg | Sum or average of a numeric field value matches condition.<br>**Example:** volume of data transferred exceeded specific value. |
| value_percentile | Some value appeared in the defined percentile.<br>**Example:** process run on less than 1% of all hosts. |

# Anatomy of a Sigma Correlation Rule



# Extended Correlation Conditions



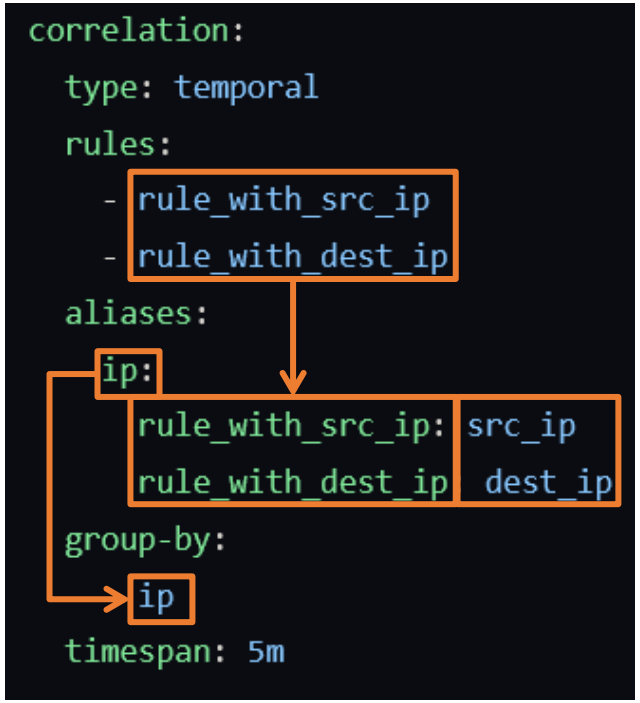
- Simple correlation conditions in temporal correlation rules were only sufficient for expression of necessary rule type counts.
  - Negations are not possible: Rule A and B trigger but not C.
  - Alternatives are not possible: Rule A and B trigger and additionally C or D
- Extended correlation conditions allow to specify such complex event relationships.
- It is not necessary anymore to specify rule lists but if one is specified, it must be consistent with the condition.

```
correlation:
  type: temporal
  timespan: 1h
  condition: rule_a and (rule_b or rule_c) and not rule_d
```

# Field Aliases



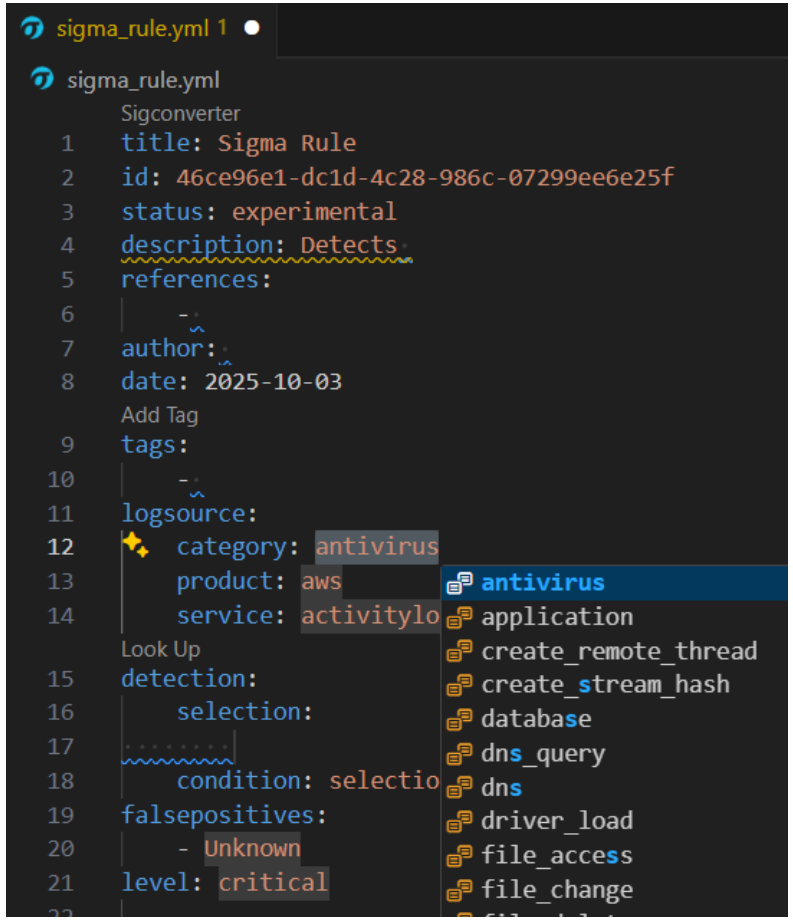
- Problem: grouping of values across different fields.
- Example: Source IP of event matched by rule A is destination IP of event matched by rule B.
- Fields aliases define a *virtual field* by mapping of rules to field names in the matched events.



# Tools

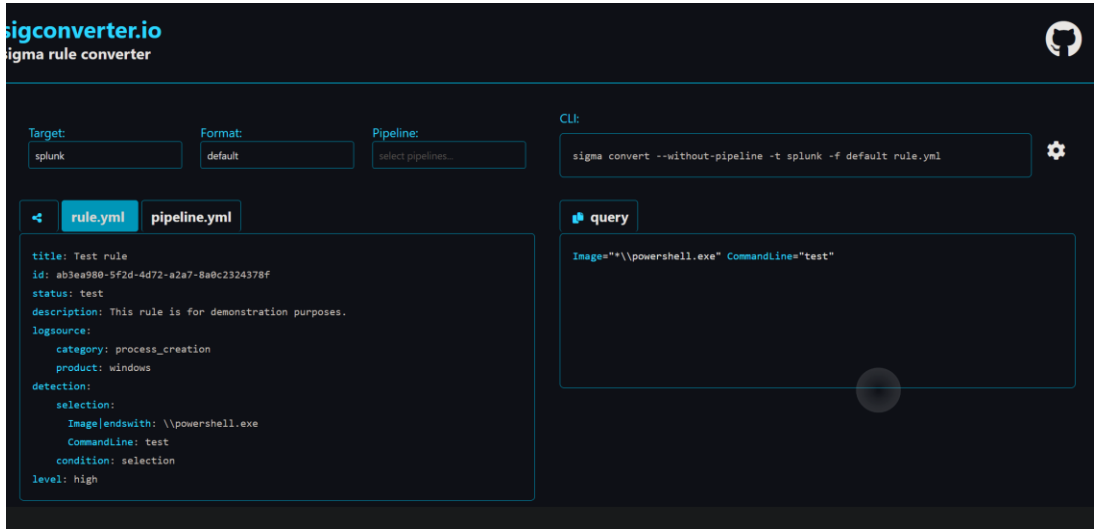
# Sigma VSCode Extension

- Sigma rule template
- Autocompletion
- Conversion with sigconverter.io directly from VSCode.



# sigconverter.io

- Webapp using pySigma in the backend for conversion.
- Supports most backends and processing pipelines.
- Shows Sigma CLI command line that can be used for conversion.
- Ideal for prototyping and quick conversion of single rules and development of a conversion configuration.



# Writing Rules with LLMs



- Prompt example: „Write Sigma rules for xxx (https://...)“
- Include checks with: “Use the installed Sigma CLI to validate the generated rules with “sigma check <rule file>”. Fix issues identified while validation.”
- Agentic mode recommended. LLM should be able to retrieve web resources like blog articles with description of attacks and conduct checks.
- The model matters! Less capable models often:
  - Create too generic FP-prone rules like execution of standard processes.
  - Hallucinate detection logic or syntactical elements that do not really exist in Sigma.
  - Don’t generate more complex rules like correlations.
  - Generate less detailed descriptions, false positives lists, tags etc.
  - Don’t follow conventions, e.g. for rule naming and modifier usage.
- Always test the generated rules!

# Comparison of LLM Results
# Bluehammer: Rapid Password Change



UUID is not random

Proper correlation rule

## GPT-4o

```
1   title: BlueHammer Rapi  assword Changes
2   id: aabbccdd-eeff-0011-2233-445566778899
3   description: Detects rapid succession of pa
4   author: Generated by GitHub Copilot
5
6   status: ex  rimental
7   logsource:
8     category: windows
9     product: windows
      Look Up
10  detection:
11    selection:
12      EventID: 4723
13      TargetUserName: 'Administrator'
14    timeframe: 1s
15    condition: selection
16  fields:
17    - EventID
18    - Target
```

Invalid logsource

Two different EventIDs. Which is the right one?

Failed attempt to build a correlation rule.

## Claude Opus 4.6

```
title: Local Account Password Reset
id: 8a4b3c2d-1e5f-4a6b-9c8d-7e0f1a2b3c4d
name: bluehammer_admin_password_reset
status: experimental
description: |
    Detects password reset even
    Used as a building block for BlueHammer rapid pa
references:
    - https://www.cyderes.com/howler-cell/windows-ze
author: Workshop
date: 2026-04-17
Add Tag
tags:
    - attack.persistence
    - attack.t1098
logsource:
    product: windows
    service: security
Look Up
detection:
    selection:
        EventID: 4724
    condition: selection
falsepositives:
    - Legitimate administrator password resets
    - Help desk operations
```

More context information

```
title: Possible BlueHammer Exploit - Rapid
id: 9b5c4d3e-2f6a-5b7c-0d9e-8f1a2b3c4d5e
status: experimental
description: |
    Detects two or more password reset ever
    within a short timespan. The BlueHammer
    password using SamiChangePasswordUser,
    then restores the original NTLM hash to
    on the same account in rapid succession
references:
    - https://www.cyderes.com/howler-cell/w
author: Workshop
date: 2026-04-17
Add Tag
tags:
    - attack.credential_access
    - attack.t1003.002
    - attack.privilege_escalation
    - attack.t1134
correlation:
    type: event_count
    rules:
        - bluehammer_admin_password_reset
    group-by:
        - TargetUserName
    timespan: 5m
    condition:
        gte: 2
level: high
```

password, use it to authenticate, then reset it back to the original hash. Security event logs should capture both password change events (Event ID 4723/4724) in quick succession - a pattern that has little legitimate explanation.

# Comparison of LLM Results
# Sensitive Registry Access on Shadow Copy

UUID is not random

## GPT-4o

```yaml
title: BlueHammer VSS Enumeration Detection
id: 12345678-1234-5678-1234-567812345678
description: Detects unexpected VSS enumeration
author: Generated by GitHub Copilot
date: 2026/04/20
status: experimental
logsource:
  category: process_creat
  product: windows
Look Up
detection:
  selection:
    Image: '*\\ntquerydirectoryobject.exe'
    CommandLine: '*HarddiskVolumeShadowCopy*'
  condition: selection
fields:
  - CommandLine
  - ParentImage
  - ParentCommandLine
  - User
falsepositives:
  - Legitimate system or backup tooling
level: high
```

This is a Windows API call, not an executable!

Doesn‘t follows convention to use contains modifier.

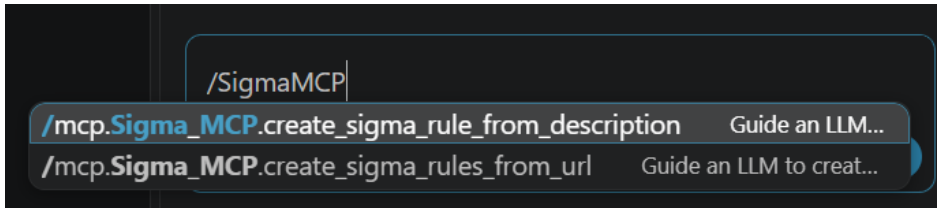
## Claude Opus 4.6

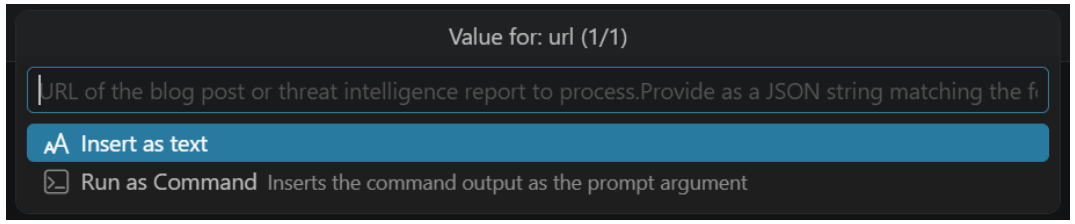
```yaml
title: Possible BlueHammer Exploit - Sensitive Registry
id: 1c6d5e4f-3a7b-6c8d-1e0f-9a2b3c4d5e6f
status: experimental
description: |
    Detects access to SAM, SYSTEM, or SECURITY registry
    device paths. The BlueHammer exploit forces Microsof
    then freezes Defender using Cloud Files callbacks an
    it reads the SAM, SYSTEM, and SECURITY hives from th
    hashes and LSA secrets for local privilege escalatio
    Requires object access auditing to be enabled.
references:
    - https://www.cyderes.com/howler-cell/windows-zero-d
author: Workshop
date: 2026-04-17
Add Tag
tags:
    - attack.credential_access
    - attack.t1003.002
    - attack.t1003.004
logsource:
    product: windows
    service: security
Look Up
detection:
    selection:
        EventID: 4663
        ObjectName|contains: 'HarddiskVolumeShadowCopy'
        ObjectName|endswith:
            - '\Windows\System32\Config\SAM'
            - '\Windows\System32\Config\SYSTEM'
            - '\Windows\System32\Config\SECURITY'
    condition: selection
falsepositives:
    - Backup software accessing registry hives from shad
    - System restore operations
level: critical
```

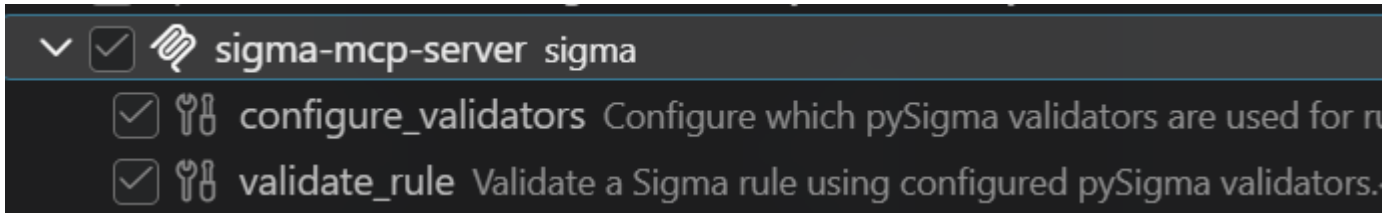
Detection of access itself instead of (non-existing) tool usage.

# Sigma MCP Server

- https://github.com/SigmaHQ/sigma-mcp-server
- Supports creation of Sigma rules with tools (validation), prompts and
- Recommended installation: pipx from local repository (no release yet)
- Used as command communicating the MCP protocol via standard I/O



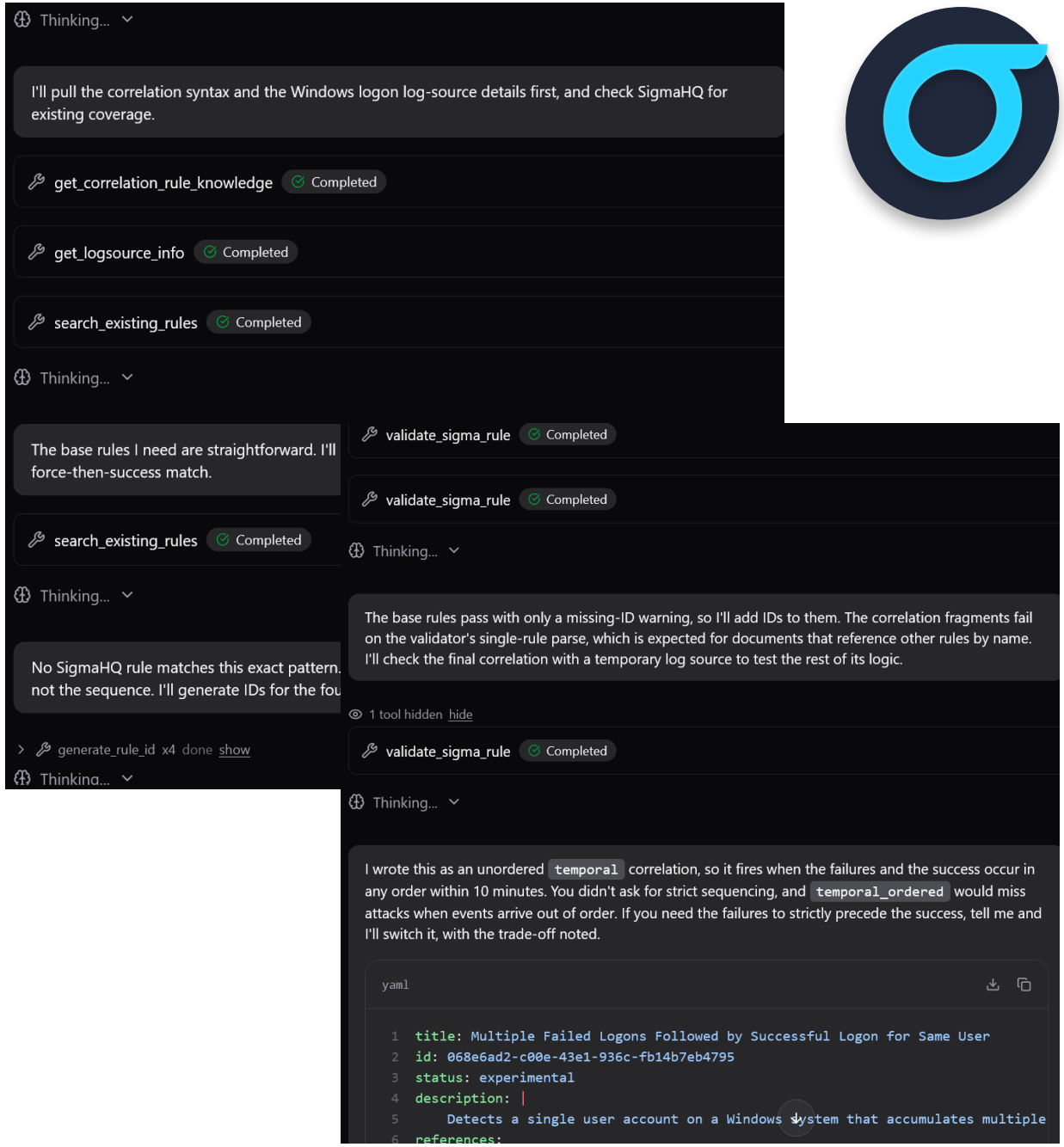



# Sigma Modeling Engine

https://github.com/thomaspatzke/simone

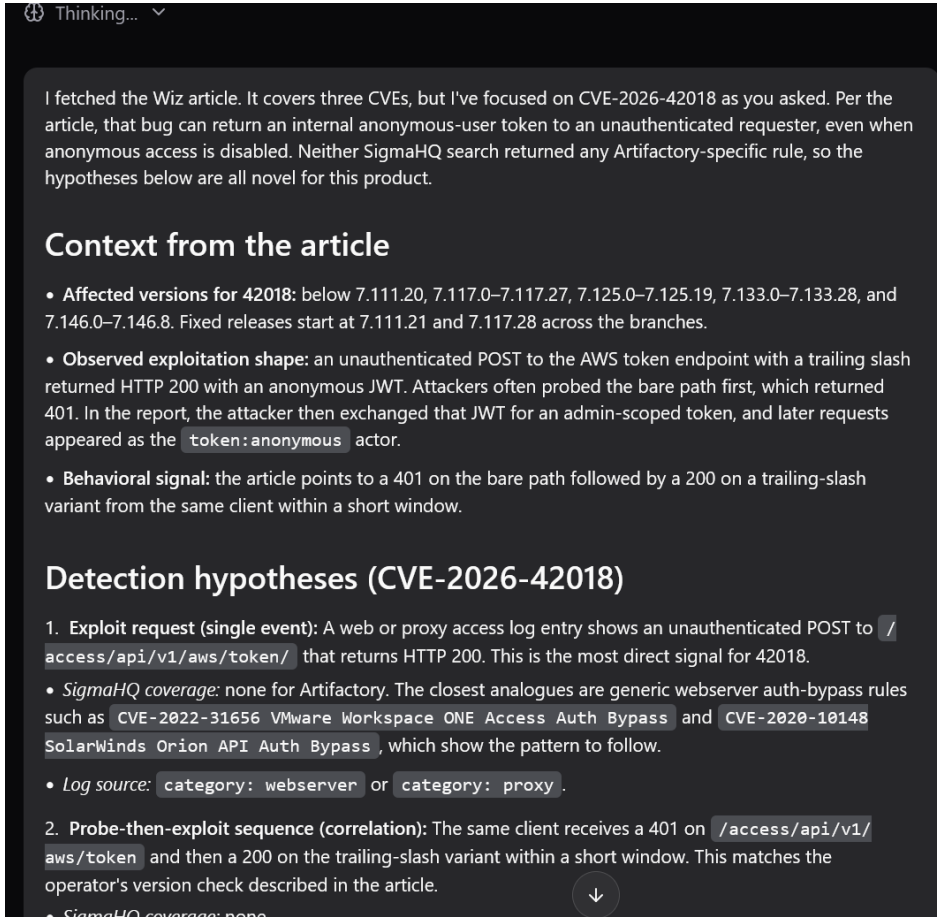
Implements Detection Engineering Workflow with Pydantic AI:

1. Analyzing Source
2. Creating Hypotheses
3. Check if Sigma rules already exist in SigmaHQ repository
4. Make suggestion to user
5. Pick appropriate log source (or let the user choose)
6. Draft rule
7. Validate with pySigma, fix, reiterate
8. Write results to an output directory



# Features

- Can fetch URLs, process PDFs and do ist research.
- UUID generation and rule validation
- SigmaHQ rule repository index & search
- Domain knowledge like log sources and best practices baked in.
- Rule output as file in configured directory.



# Experiences & Plans



- Performs quite well with cheap models (Claude Haiku, OpenAI Luna)
- Workflow from blog post to detections costs usually few Cent.
- Trying to be token-efficient, e.g. correlation rule knowledge or log sources only loaded on demand into context.
- Idea: expose SIEM for deep rule validation

**How the logic works**

- Rule 1 flags the exploit request itself: a POST to the trailing-slash path that returns 200. This is the signal the article describes for CVE-2026-42018.
- Rule 2 looks for the probe pattern: a 401 on the bare path followed by a 200 on the trailing-slash path from the same client IP within five minutes. I used `temporal`, not `temporal_ordered`, so out-of-order log arrival won't cause misses.

**Assumptions to check**

- The article doesn't state the HTTP method for the 401 probe. I assumed POST to match the exploit request.
- The `cs-uri-stem` field assumes the path is at the root. If Artifactory sits behind a context path (for example `/artifactory/...`), the `contains` match still works, but the `endswith` match in the probe rule should be checked.
- `c-ip` is the TCP peer address. If a reverse proxy or load balancer sits in front of Artifactory, every client will share one IP and the grouping breaks. In that case, group by the forwarded client address instead.

**Known false positives and tuning**

- Rule 1 can fire on legitimate authenticated clients that use the trailing-slash path. If your logs record the username, exclude known service accounts.
- Rule 2 can fire on scanners that test endpoint variants. Exclude internal vulnerability scanners by source IP.

**Next steps**

- The article also describes post-exploitation signals, such as `PUT /api/security/users/` returning

# Some Notes on AI-Generated Rules



- Always validate! AI results in rule generation got much better but detections are very sensitive to false positives/negatives
- SigmaHQ rule repository focuses on quality than mass.
- Each pull request is checked manually (at least to a certain degree)
- Imbalance: few minutes creation of AI slop can bind lot of maintainer time.
- Add meaningful description of PR and its content. Why does the PR exist?
- Please validate your rules first before doing the pull request!
- Don‘t flood with pull request, preferred less larger than many small pull requests.

# Recap & Outlook

- In this workshop we learned…
  - What a Sigma rule is and why it makes sense to use them.
  - Writing Sigma rules for single events.
  - Writing Sigma correlation rules.
  - Using LLMs for writing rules.
- Another topic: operationalization
  - Conversion
  - Adapting to environment
  - CI/CD

Sigma operationalization workshop
https://github.com/thomaspatzke/sigma-workshop-operationalization

# Further Trainings



**Writing and converting Sigma rules:**

Applied Network Defense: Detection Engineering with Sigma

- https://www.networkdefense.co/courses/sigma/



**Building a detection pipeline with Sigma:**

Taggart Institute: Automated Detection with Sigma

- https://taggartinstitute.org/p/detection-with-sigma



# Further Resources



- SigmaHQ (sigmahq.io)
- SigmaHQ Discord
- SigmaHQ GitHub
- thomas@patzke.org

SigmaHQ

Discord

Rule Repository

Sponsoring

# Exercises

# Exercise 1: Failed Logon



- Write a Sigma rule that detects failed logon attempts in Windows security logs.
- Some hints:
  - The Windows EventID is 4625
  - MITRE ATT&CK technique T1110.001 (Brute Force: Password Guessing) fits quite well.

# Exercise 2:
# Write a Sigma Rule for PRT Token Theft

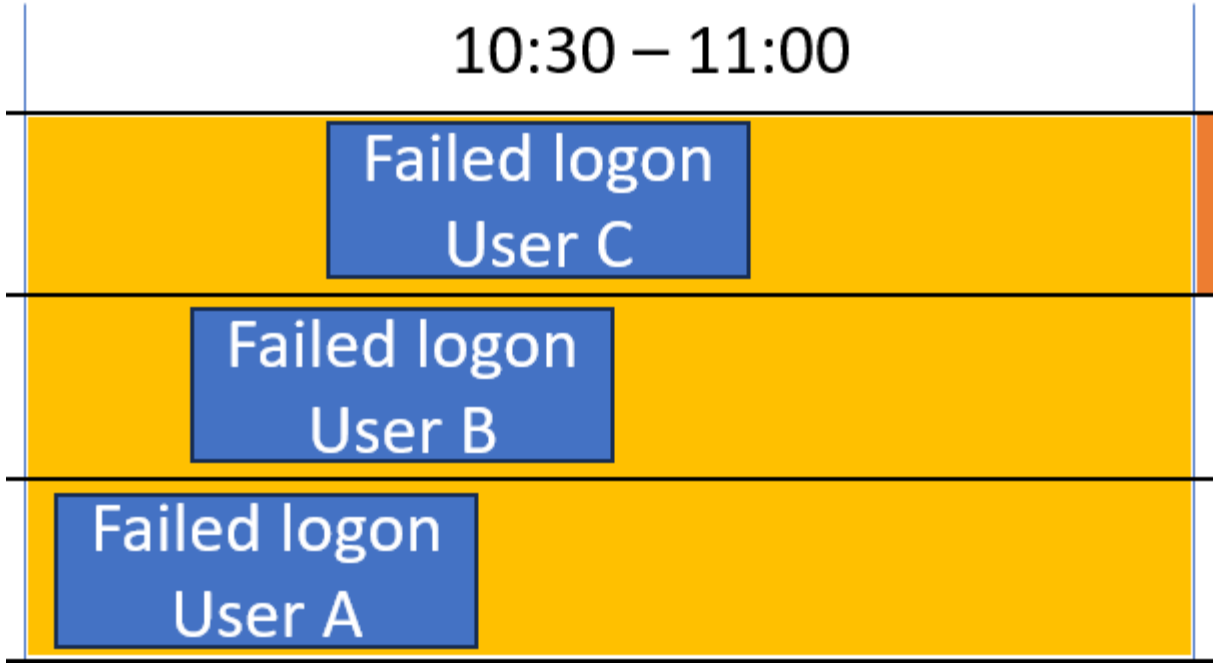


- Process creation of BrowserCore.exe
- Only allow the following parent images:
  - cmd.exe
  - Teams.exe
- Only allow invocation that contain a specific Chrome plugin: chrome-extension://ppnbnpeolgkicgegkbkbjmhlideopiji/
- MITRE ATT&CK techniques are T1003, T1134.001

# Exercise 3:
# Write a Password Spraying Rule



- Password spraying = trying passwords across many users (from single source)
- The source IP is located in the field IpAddress



# Exercise 4:
# Write a Temporal Correlation Rule



- The rule should match on the following events:
  - Failed logon
  - Successful logon
  - PRT theft
- For a single user
  - For successful and failed logons it's contained in TargetUserName.
  - For the process creation in User.
- Two event types are sufficient.

